PixelMint

Product Requirements Document (PRD) - Decentralized NFT Marketplace

Field	Details
Project Name	PixelMint (Web3 NFT Marketplace & Creator Studio)
Prepared By	Aditya Nair - Aspiring Product Manager
Document Version	1.0 (Baseline Project & Feature Specification)
Status	Portfolio / Functional Front-End Prototype Documentation
Date	October 2026

Document Purpose & Methodology: This PRD was created as a capstone product management exercise based on the actual, working codebase of PixelMint. It treats the codebase as the sole source of truth. Every functional capability, mock simulation, and architectural gap is documented transparently without exaggerated corporate buzzwords or fabricated metrics. Its objective is to demonstrate fundamental product thinking: user problem framing, persona definition, user journeys, MVP scope rationalization, functional requirements, edge case identification, risk assessment, and the critical distinction between product design and engineering implementation.

Table of Contents

S.No.	Section	Summary of Contents
1.	**Product Overview**	Context, problem space, user capabilities, and project origin
2.	**Problem Statement**	Web3 onboarding complexity, opacity, gas fees, and problem classification
3.	**Product Vision**	Vision statement and core guiding product principles
4.	**Target Users & Personas**	Detailed profiles for Creator Elena, Collector Marcus, and Explorer Sophia
5.	**User Problems and Needs**	Comprehensive mapping of pain points, user needs, and product responses
6.	**Product Goals**	Five measurable and realistic product goals for PixelMint
7.	**User Journeys**	Step-by-step user journey maps across key marketplace flows
8.	**Product Scope**	Clear demarcation of In-Scope prototype features vs. Out-of-Scope future roadmap
9.	**Feature Overview**	MoSCoW prioritized feature inventory with current implementation status
10.	**MVP Definition**	Minimum Viable Product boundaries and component-by-component rationale
11.	**Functional Requirements**	Formal specifications from FR-01 to FR-18 with priority and verification notes
12.	**User Stories**	14 persona-aligned user stories following industry standard formatting

13.	**Acceptance Criteria**	Explicit testable criteria across search, minting, wallet, checkout, and bidding
14.	**Non-Functional Requirements**	Usability, performance, security communication, reliability, and error handling
15.	**UX / UI Requirements**	Detailed breakdown of screens, layouts, component interactions, and modals
16.	**Edge Cases and Error States**	10 realistic edge case scenarios with system responses
17.	**Proposed Success Metrics**	Unmeasured product success framework spanning acquisition, conversion, and retention
18.	**Risks and Mitigations**	Assessment of smart contract, wallet UX, fee volatility, and regulatory risks
19.	**Assumptions**	Product, user, and technical assumptions underlying this specification
20.	**Open Product Questions**	Strategic questions regarding Layer-2 networks, custody, and economics
21.	**Future Opportunities**	AI-assisted NFT description generation and creator studio tooling
22.	**Implementation Status Audit**	Brutally honest feature audit comparing expected behavior vs. codebase reality
23.	**Product Roadmap**	Three-phase evolutionary roadmap from prototype to testnet and production
24.	**Conclusion & PM Retrospective**	Personal learnings, trade-offs made, and portfolio summary

25.	**Appendix**	Project architecture, repository structure, and Web3 terminology glossary

1. Product Overview

PixelMint is a prototype for a decentralised digital art marketplace and creator studio with the aim of making it easier to discover, mint and trade non-fungible tokens (NFTs). It was developed as part of a university final-year project and was intended as a way of examining how the complicated and intimidating user journeys which are usual in early Web3 applications can be simplified into intuitive and accessible consumer experiences.

The project is dealing with the issues involved in digital collectibles and the creator economy. Although platforms like OpenSea, LooksRare, and Magic Eden have set the technical standard for the exchange of decentralised assets, they usually pose considerable usability problems for users who are not technically minded. New users are frequently faced with confusing jargon, multiple-step signature requests, unpredictable network charges ('gas'), and disjointed creator tools. PixelMint aims to reduce these obstacles by introducing clearer user flows, transparent price summaries, and safe transaction simulation safeguards.

1.1 Core Capabilities Built in PixelMint

Within the current functional application, users can perform the following key actions:

- **Discover Curated Artworks:** Browse a catalog which includes one-of-one digital artworks organised by the categories of Art, Collectibles, Photography, Music, Gaming, and Virtual Worlds.
- **Discovery Controls:** Multi-Parametric Search and Filtering: Assets can be dynamically filtered using a search query, the sale method (Fixed Price or

Timed Auction), a price range slider (0 to 15 ETH), and the creator's verified status.

- **Token Provenance:** Check the Provenance and the On-Chain Metadata: you can view the smart contract specifications at the item level, the token standards (ERC-721), the token IDs, the creator royalty splits, the trait rarity attributes, and the full historical transaction logs.
- **Wallet integration:** simulate a connection with multiple Web3 wallets: connect the simulated provider interfaces (MetaMask, WalletConnect, Coinbase Wallet, Phantom, Rainbow) and switch between the different network contexts for Ethereum, Polygon, Arbitrum, and Solana.
- **Self-serve minting:** In Creator Studio and Minting you can upload your custom artwork files (using the file picker or by dragging and dropping them), or select generative presets, set the token attributes, configure the secondary royalties (up to 20 per cent), estimate the gas costs, and then deploy the newly minted items into the active catalog.
- **Direct Buy:** Instant checkout and atomic purchasing, which involves buying assets at a fixed price together with a breakdown of the item price and gas fee, carrying out a balance check, simulating the blockchain execution, transferring ownership, and logging the activity.
- **Bidding in an auction:** When taking part in a live timed auction you can place competitive bids on artworks that are listed for auction, with the bid increments having a minimum value of +0.1 ETH, checks being made of your balance, and the timeline of your bidding activity being recorded.
- **Offer Negotiation:** Make Direct Offers: Put forward purchase suggestions not on the open market together with specific price figures and set expiry periods (of 1, 3, or 7 days) directly with the asset owners.
- **Inventory Management:** On the Collector Portfolio Dashboard you can monitor the assets you have collected, the drops you have created, the items you have favourited, the net valuation of your portfolio in ETH, and also access an interactive feed of transaction activities.
- **Risk-free testing:** With the integrated test faucet, you can claim 5.0 simulated ETH with just one click in order to easily test the buying and bidding processes without any personal financial risk.

1.2 Project Origin & Context

I decided to make PixelMint my final-year project since the combination of digital property rights, creator royalties, and the consumer user experience involves one of the most difficult areas for product design in today's software. Instead of creating a standard web application, I aimed at working in a field where users actually experience considerable anxiety and technical difficulties. By building the application myself, I was able to see how state management, wallet signatures, on-chain metadata, and the marketplace mechanisms interact - and also where the consumer user experience tends to fail.

2. Problem Statement

The blockchain technology enables provable digital ownership and automatic payment of royalties to creators, but widespread acceptance by ordinary consumers is still greatly hindered by difficulties in the user experience. As a result of comparing the various existing marketplaces and carrying out informal walkthroughs with my university peers, four main problem areas were found:

2.1 Friction Areas Observed

1. **High Difficulty of Entry into Web3 Ecosystem and Mission Creep:** Traditional NFT platforms presume that users have a high technical level. First-time visitors are thrown into the dizzying world of cryptic hexadecimal addresses, sudden network switches, and signature requests that seem like software fails instead of assurances of safety.

2. **Unstable Cost of Transactions and Gas Worries:** When buying an asset on-chain, a user faces the problem of constantly changing fees that differ every second. Most marketplaces wear out users with vague pricing statements and barely inform them about the cost of the purchase until it is completed.

3. **Distorted Browsing Experience and Low Usability of Websites:** Visiting huge NFT marketplaces most of the time is not a pleasant experience. The sites are cluttered with low-quality collections, repeated items, and impossible filters,

making it nearly impossible for users to trace the authenticity and the history of products.

4. **Difficult Crafting Process for Creators' Communities:** Converting digital images into tokenized assets usually requires technical literacy on behalf of independent artists, they need to know how to use IPFS storage, deploy a smart contract, and understand royalty management principles (ERC-2981).

2.2 Problem Classification Matrix

In order to ensure product integrity, the table below presents clear distinctions between the issues observed, solutions implemented in the PixelMint prototype, and the unresolved issues for the future engineering work.

Problem Area	Observed Friction	PixelMint Prototype Response	Status / Implementation Reality
Web3 Onboarding	Users fear signing unknown wallet prompts or losing funds during wallet setup.	Simulated multi-wallet modal with 1-click connection, test faucet (+5 ETH), and network switcher.	Functional UI simulation. Does not invoke real external browser wallet extensions.
Transaction Opacity	Checkout screens hide gas fees, resulting in transaction failures due to balance shortfalls.	Instant Checkout modal with explicit line-item breakdown (Asset Price + ~0.0035 ETH Gas	Fully implemented logic. Calculates balance sufficiency and updates local state.

		+ Royalty split = Total Cost).	
Gas Fee Reduction	High Layer-1 Ethereum mainnet gas fees create a barrier to entry for micro-transactions.	Visual gas fee estimation (~0.012 ETH mint, ~0.0035 ETH buy) displayed upfront.	Open Consideration. Gas reduction was an initial goal; no Layer-2 rollup or gasless contract is currently deployed.
Creator Complexity	Creators struggle with token metadata, trait schemas, and royalty definitions.	Step-by-step Creator Studio with drag-and-drop upload, preset picker, royalty slider (0-20%), and custom trait builders.	Fully functional client-side minting flow. Stores metadata in application state & localStorage.
Security & Trust	Pervasive fear of malicious contracts, phishing links, and unverified smart contracts.	Clear provenance tabs showing Token ID, ERC-721 standard, verified artist badges, and historical logs.	UI Prototype. '100% Audited' badge on landing page is a marketing placeholder; no formal audit was performed.

Important Product Note on Gas Reduction & Security Audits: As an aspiring Product Manager, I believe in complete transparency regarding product maturity. Gas fee reduction was an essential motivating problem for PixelMint, but a production Layer-2 or meta-transaction solution (e.g., EIP-2771 gasless relayer or Arbitrum rollups) was NOT built into this version. Similarly, no third-party security audits (e.g., OpenZeppelin or CertiK) were conducted. These remain documented engineering requirements for subsequent production phases.

3. Product Vision

Product Vision Statement: "To develop an uncomplicated, transparent, and easy-to-use digital asset platform that allows creators to mint their artworks and collectors to buy and sell NFTs without any confusing elements or steps related to Web3 technology."

Unlike other companies, PixelMint does not try to create a revolutionary blockchain protocol; instead, it is focused on simplifying the blockchain process. We picture a world in which digital artists can prove their ownership of artworks in a matter of three minutes while collectors can conduct transactions confidently knowing about the total price of assets, their origin and rights of ownership.

3.1 Core Guiding Principles

1. **Simple Language Matters:** It is essential that every Web3 principle utilizes language that is easy to understand. Jargon like 'slippage tolerance' and 'calldata' should not appear in consumer interfaces.

2. **Clear Costs:** There should be no hidden costs involved. The checkout and mint process must allow users to see in complete clarity the breakdown of the price, gas estimates, and the amount paid as the royalty before confirming their purchase.

3. **Freedom of Choice:** Artists should not be required to write any code to be able to sell their artworks. All relevant details can be dealt with using user-friendly forms.

4. **Safety First:** Thoroughly enable features that will allow users to play around and learn how to use the market effectively without risking their own money.

4. Target Users & Persona Profiles

PixelMint is designed around three primary user archetypes. Rather than relying on generic demographic generalizations, these personas focus on user goals, technical comfort levels, and behavioral pain points.

4.1 Persona 1: Elena Vance - The Independent Digital Creator

- **Background:** Freelance digital illustrator and generative artist with 4 years of experience selling prints via Etsy and Instagram.
- **Primary Goal:** Wants to monetize high-resolution digital art directly, protect intellectual property, and earn automated royalties from secondary market sales.
- **Key Frustrations:** Overwhelmed by smart contract development, gas fee fluctuations, IPFS pinning, and configuring complex ERC-721 metadata attributes.
- **What Elena Needs from PixelMint:** A clean, visual minting studio where she can upload image files, choose a preset, set a 10% royalty split with a slider, select a fixed price or auction, and deploy in one click.

4.2 Persona 2: Marcus Chen - The Digital Art Collector

- **Background:** Tech-savvy professional who owns cryptocurrencies (ETH, SOL) and follows emerging contemporary digital artists on Twitter/X.
- **Primary Goal:** Discovers unique 1-of-1 digital artworks, verifies authenticity and creator identity, and acquires high-potential assets at fair market prices.

- **Key Frustrations:** Encountering fake listings, lack of transparent collection volume data, confusing bid increments, and unexpected transaction failures at checkout.
- **What Marcus Needs from PixelMint:** Instant checkout with transparent gas estimates, verified artist badges, rich attribute viewing, and a dedicated personal profile showing net portfolio valuation and collection history.

4.3 Persona 3: Sophia Patel - The Web3 Beginner / Explorer

- **Background:** College student and digital art enthusiast who has heard about NFTs in news media and design courses but has never made a blockchain transaction.
- **Primary Goal:** Wants to explore what NFTs are, browse aesthetic artworks, and understand how digital ownership works without risking real personal savings.
- **Key Frustrations:** Intimidated by seed phrases, terrified of clicking 'Confirm' on MetaMask, and confused by cryptocurrency exchange transfers and gas terminology.
- **What Sophia Needs from PixelMint:** A safe, frictionless sandbox experience where she can connect a simulated wallet, claim free test ETH from a faucet, experiment with buying or bidding, and see immediate visual feedback.

5. User Problems and Needs Matrix

The following matrix aligns observed user problems across our three target personas with specific user needs, the resulting product responses engineered into PixelMint, and an honest assessment of current prototype implementation.

Target User	Observed Problem	Underlying User Need	PixelMint Product Response	Prototype Reality
Creator (Elena)	Does not know how to write Solidity smart contracts or upload files to IPFS.	Code-free artwork publishing tool with automated token standard configuration.	Creator Studio with drag-and-drop file upload, automated ERC-721 token formatting, and metadata builder.	Functional UI. Stores base64 / URLs in local state.
Creator (Elena)	Lacks control over secondary sale compensation on open marketplaces.	Predictable, customizable creator royalty split enforced at token creation.	Configurable royalty slider (0% to 20%) during the minting process, saved in token attributes.	Implemented in data model and checkout breakdown.

Buyer (Marcus)	Checkout pages obscure network fees until wallet confirmation popups appear.	Complete upfront cost certainty prior to initiating any transaction signature.	Instant Checkout modal showing item price, estimated gas fee (~0.0035 ETH), creator royalty, and total cost.	Fully implemented with balance check & deduction.
Buyer (Marcus)	Difficult to assess whether a listed item is genuine or from a copycat collection.	Visual trust signals and immutable provenance history on asset pages.	Verified creator badges, collection volume rankings, and complete chronological transaction history tabs.	Fully implemented in UI with mock history logs.
Beginner (Sophia)	Fear of financial loss while learning how Web3 marketplace mechanics work.	Risk-free simulation environment with guided actions and test funds.	1-click test faucet (+5.0 ETH), simulated multi-wallet connection, and educational	Fully implemented. Allows complete end-to-end sandbox testing.

			toast notifications.	
Beginner (Sophia)	Unclear whether transactions succeeded, failed, or are stuck pending on-chain.	Immediate, friendly feedback on transactional status without cryptic hash strings.	Multi-stage modal progress states (Checkout -> Processing -> Confirmed) paired with global toast alerts.	Fully implemented via asynchronous simulation.

6. Product Goals

To guide product decisions and prioritize development efforts, five realistic product goals were established. These goals focus on usability, transparency, and architectural clarity rather than speculative commercial vanity metrics:

- **Goal 1: Simplify Digital Asset Discovery:** Enable users to browse, search, and filter a diverse catalog of digital assets within 2 clicks from the homepage, providing instant filtering by price, category, sale type, and creator verification without page reloads.
- **Goal 2: Deliver Transparent Transaction Economics:** Ensure 100% of purchase, bidding, and minting flows display upfront line-item cost summaries including estimated network fees and creator royalties - eliminating unexpected fee surprises at the point of confirmation.
- **Goal 3: Streamline 1-of-1 NFT Creation and Listing:** Provide an intuitive, code-free creator studio that enables digital artists to configure token title, description, category, pricing structure, royalties, and custom trait attributes in under 3 minutes.

Goal 4: Demystify Wallet Onboarding via Safe Simulation: Offer a safe, educational onboarding experience where first-time Web3 explorers can connect a simulated wallet, switch networks, and claim test funds with zero setup friction and zero personal financial risk.

Goal 5: Provide Complete Item Provenance & Traceability: Maintain clear provenance tracking for every asset by recording minting events, direct sales, bids, and owner transfers in an accessible, readable transaction timeline on the item detail page.

7. User Journeys

The following step-by-step user journeys document how key personas navigate PixelMint to accomplish their primary goals. Each journey maps the current functional flow built into the application codebase.

7.1 Journey A: First-Time User Exploring the Marketplace (Sophia)

Step 1 (Landing): Sophia arrives at PixelMint landing page (HomePage.tsx), views live market pulse ticker, hero drop, and curated statistics.

Step 2 (Explore Navigation): Clicks 'Explore Marketplace' button in hero section or 'Explore' link in fixed Navbar.

Step 3 (Category Filtering): Browses catalog grid in ExplorePage.tsx. Clicks category pill 'Collectibles' to filter items.

Step 4 (Search Query): Types 'Cyber' into the search input. The catalog dynamically updates in real time to show matching items.

Step 5 (Advanced Refinement): Click 'Filters' to open the advanced drawer. Adjusts price range slider to maximum 5 ETH and selects 'Price: Low to High' from sort dropdown.

Step 6 (Item Inspection): Clicks on 'Pixel Punk #1337' card to navigate to NFTDetailPage.tsx to examine full artwork, traits, and historical sales.

7.2 Journey B: NFT Buyer Completing a Fixed-Price Purchase (Marcus)

Step 1 (Item Selection): Marcus discovers 'Cosmic Dreamer #42' listed at 2.45 ETH on NFTDetailPage.tsx.

Step 2 (Provenance Check): Reviews item metadata: verifies creator is 'CryptoArtist' (Verified badge), checks ERC-721 token standard, 10% royalty split, and previous sale history.

Step 3 (Wallet Prompt): Click the 'Buy Now' button. The system detects that the wallet is disconnected and automatically opens WalletModal.tsx.

Step 4 (Wallet Connection): Selects 'MetaMask'. Modal displays brief spinner, completes simulated connection, updates Navbar with wallet address and 14.85 ETH balance.

Step 5 (Cost Breakdown Review): BuyModal.tsx opens with item preview, base price (2.45 ETH), estimated gas fee (~0.0035 ETH), royalty split, and total cost (2.4535 ETH).

Step 6 (Balance Verification): System verifies Marcus's balance (14.85 ETH > 2.4535 ETH). The balance indicator glows green.

Step 7 (Execution): Marcus clicks 'Confirm & Pay'. Modal advances to 'Processing on Chain' state with loading spinner.

Step 8 (Settlement & Feedback): Purchase completes: 2.4535 ETH is deducted, item ownership transfers to Marcus, new 'Sold' transaction is prepended to history, new activity is logged, and a success toast notification appears.

7.3 Journey C: NFT Creator Minting and Listing a New Artwork (Elena)

Step 1 (Studio Access): Elena connects her wallet via Navbar and clicks 'Create' in navigation to open CreatePage.tsx.

Step 2 (Artwork Upload): Drags and drops high-resolution PNG artwork onto the upload dropzone. System renders immediate visual preview.

Step 3 (Metadata Entry): Enters NFT title ('Neon Dimension #01') and detailed artistic description.

Step 4 (Classification): Selects category ('Art') and target blockchain ('Ethereum').

Step 5 (Pricing & Royalties): Selects listing format: clicks 'Fixed Price' and enters 1.5 ETH. Adjusts royalty slider to 10%.

Step 6 (Trait Configuration): Adds custom on-chain traits: clicks 'Add Trait' and defines 'Atmosphere: Cyberpunk' and 'Rarity: Genesis Drop'.

Step 7 (Mint Submission): Reviews gas estimate (~0.012 ETH) and clicks 'Deploy & Mint NFT'.

Step 8 (Mint Confirmation): System validates required fields, deducts 0.012 ETH simulated gas, creates new NFT object, prepends it to the global catalog, logs 'Minted' activity, and displays celebration confirmation screen with direct link to view listing.

7.4 Journey D: Timed Auction Bidding Flow

Step 1 (Auction Discovery): User navigates to an auction item (e.g., 'Abstract Genesis' with highest bid 4.80 ETH).

Step 2 (Bid Initiation): Clicks 'Place Bid'. BidModal.tsx opens showing current highest bid and minimum valid increment (4.90 ETH).

Step 3 (Bid Adjustment): User uses quick increment buttons (+0.1, +0.5, +1.0) or types custom bid of 5.50 ETH.

Step 4 (Validation): System validates bid is higher than 4.80 ETH and verifies user balance is sufficient.

Step 5 (Bid Settlement): User submits bid. System updates highest bid, adds bid to bid history array, logs 'Bid Placed' user activity, and displays success toast.

7.5 Journey E: Submitting an Off-Market Offer

Step 1 (Offer Initiation): User views an item owned by another collector and clicks 'Make Offer' on NFTDetailPage.tsx.

Step 2 (Offer Form): MakeOfferModal.tsx opens displaying listing price and owner name.

Step 3 (Terms Selection): User inputs offer an amount in ETH (e.g., 2.0 ETH) and selects expiration duration (1 day, 3 days, or 7 days).

Step 4 (Offer Dispatch): User clicks 'Submit Offer'. System validates amount, generates pseudo transaction hash, records 'Offer Made' in user activity feed, and notifies user via toast.

8. Product Scope

One of the main responsibilities in product management is strictly defining the boundaries of the scope. In executing the project, the features were classified into two distinct groups: in-scope (that were implemented in the current prototype) and out-of-scope (that will be built in the future).

8.1 In Scope (Current Prototype Implementation)

• **Marketplace Discovery:** Fully functional catalog browsing using real-time responsive grid (one can toggle between 3/4 columns).

• **Search Engine:** Keyword search for title, creator, collection, and category with dropdown for suggested keywords.

• **Catalog Improvement:** Any combination of multiple filters by category type, selling mode (either Fixed or Auction sale), price range, verified status, and ordering.

• **Item Presentation:** Large item view with item image view, metadata, pricing display, and transaction history options.

• **Web3 Connection:** Connection using multiple web3 providers (MetaMask, WalletConnect, Phantom, Coinbase, Rainbow) with network switch.

• **Testnet Simulation Faucet:** Single-click faucet to provide +5 ETH for transactions.

• **Creator Studio:** Minting studio where artists load their file with forms, drag-and-drop images, preset art templates, traits, and royalties management.

• **Atomic Purchase:** Instant checkout with gas costs, balance check, etc.

• **Auction Bid:** Live auction bid with dynamic increase buttons.

• **Offer Submission:** Window to submit custom offer amounts.

• **Profile and Inventory:** Profile showing owned pieces of art and drops created by the collector.

• **Collections Ranking:** List of the best collections with their floor price, daily volume, etc.

8.2 Out of Scope / Future Considerations

- Real-Time Smart Contract Deployment: Deployment of fully operational smart contracts developed using Solidity on public test networks like Sepolia/Goerli and the Ethereum mainnet.

- Authentic Web3-Integrated Providers: Seamless connectivity with browser extension RPCs using technologies like ethers.js, viem, or wagmi.

- Decentralized Storage Pinging: Enable seamless storage using IPFS/Filecoin decentralization or Arweave nodes through conventional pinning (base64 data URLs/Https is currently used).

- Layer-2 Gasless Rollups Technology: Utilizes fully functional gasless metatransactions like EIP-2771 and Biconomy on layer-2 state transitions and automated Layer-2 bridge.

- AI-Powered Assistance to Creators: Technology enables images uploaded by creators to be described through computer vision and AI.

- Onboarding Fiat Currency: Technology that allows payments via credit card and Apple Pay.

- Off-Chain Bookkeeping or Escrow Deal: Provides sellers with an opportunity to accept or counter offers.

- Native mobile applications: Technology provides the mobile application of iOS and Android (mobile optimized web interface).

9. Feature Overview Matrix

The following table provides a comprehensive inventory of PixelMint's features, organized by user persona, MoSCoW priority, and verified implementation status based on codebase inspection.

Feature	Core Purpose	Target User	Priority	Implementation Status
Explore Catalog	Browse active digital artworks in a responsive visual grid	All Users	Must Have	Implemented (Functional)
Real-time Search	Search assets by title, artist, category, or collection	Buyer / Collector	Must Have	Implemented (Functional)
Dynamic Filters	Filter by category, price slider, sale type, and verification	Buyer / Collector	Must Have	Implemented (Functional)
NFT Detail Page	Inspect full asset metadata, attributes, and	Buyer / Collector	Must Have	Implemented (Functional)

	provenance history			
Wallet Modal	Simulate connecting Web3 wallets and network switching	All Users	Must Have	Implemented (Simulated)
Testnet Faucet	Claim test ETH (+5 ETH) to safely evaluate marketplace flows	Beginner / Explorer	Must Have	Implemented (Functional)
Creator Studio	Upload artwork, set price, traits, royalties, and mint token	Creator / Artist	Must Have	Implemented (Functional)
Instant Buy Checkout	Execute fixed-price purchase with gas fee breakdown	Buyer / Collector	Must Have	Implemented (Functional)
Auction Bidding	Place competitive bids on timed auction listings	Buyer / Collector	Should Have	Implemented (Functional)

Make Offer	Submit price proposals with expiration to asset owners	Buyer / Collector	Should Have	Implemented (Functional)
Profile & Inventory	Track owned, created, favorited NFTs and total valuation	Collector / Creator	Should Have	Implemented (Functional)
Collections Board	View top collections ranked by volume, floor, and change	Collector / Buyer	Should Have	Implemented (Functional)
State Persistence	Persist mints, buys, bids, and profile edits to localStorage	All Users	Must Have	Implemented (Functional)
Real Web3 RPCs	Execute real on-chain transactions via browser wallet extensions	All Users	Could Have	To Be Verified / Future
IPFS File Storage	Pin asset images and JSON metadata to	Creator / Artist	Could Have	Not Implemented / Future

	decentralized storage			
AI Description Tool	Automatically generate draft NFT descriptions from uploaded art	Creator / Artist	Future	Not Implemented / Future
Fiat On-Ramp	Purchase NFTs using credit cards without holding cryptocurrency	Beginner / Explorer	Future	Not Implemented / Future

10. MVP Definition & Scope Rationale

An MVP should represent the smallest conceivable version of the product that delivers meaningful value and validates core assumptions. For PixelMint, the MVP was strictly defined around the core transaction loop: Discover -> Connect -> Mint -> Buy.

10.1 Features Included in the MVP & Strategic Rationale

1. **Marketplace Discovery & Catalog (Must Have):** Without an aesthetic, responsive catalog, users cannot discover assets. A marketplace has zero utility without an inventory display.
2. **Keyword Search & Price Filtering (Must Have):** Users need to locate specific artists and filter by price affordability. Basic keyword search and price bounding are critical to reduce bounce rates.

3. **NFT Detail View & Metadata Presentation (Must Have):** Digital assets require provenance proof. Displaying token standards, creator verification, trait rarity, and description is required for purchase consideration.
4. **Simulated Wallet Connection & Test Faucet (Must Have):** Web3 transactions require an identity. A simulated multi-wallet interface and 1-click test faucet allow users to test transactions without financial risk.
5. **Guided Creator Studio (Must Have):** A marketplace cannot grow without supply. Creators must be able to upload images, set prices, and mint assets without touching smart contract code.
6. **Instant Checkout / Direct Buy (Must Have):** The core transaction that proves marketplace mechanics. Must validate wallet balance, display gas estimates, transfer ownership, and update state.

10.2 Features Excluded from MVP (Deliberate Trade-Offs)

- **Timed Auctions (Pushed to Phase 2):** Timed auctions introduce complex asynchronous state management (countdown timers, reserve prices, bid cancellation). The fixed price was sufficient to validate transaction demand.
- **Off-Market Offers (Pushed to Phase 2):** Bilateral offer negotiations require off-chain signature orderbooks or escrow contracts. Fixed checkout was prioritized.
- **AI Description Generation (Pushed to Phase 3):** While AI-assisted descriptions represent an exciting creator tool, it is a secondary value-add. The core minting engine had to function first.
- **Fiat Currency Processing (Pushed to Future):** Adds payment gateway complexity, KYC verification, and chargeback risks. Crypto-native simulation was prioritized for the initial scope.

11. Functional Requirements

The functional requirements below define the specific capabilities of PixelMint. Each requirement includes a unique identifier, detailed specification, priority, and verified implementation status in the codebase.

ID	Feature Name	Requirement Specification	Priority	Implementation Status
FR-01	Marketplace Catalog	The system shall display active NFT listings in a responsive grid with image, title, creator, price, likes, and sale format.	Must Have	Implemented (Functional)
FR-02	Instant Search	The system shall allow users to search assets by title, creator name, collection, and category with real-time dynamic matching.	Must Have	Implemented (Functional)

FR-03	Category Filtering	The system shall filter the catalog across predefined categories: All, Art, Collectibles, Photography, Music, Gaming, Virtual Worlds.	Must Have	Implemented (Functional)
FR-04	Price Range Filter	The system shall provide an interactive slider allowing users to filter items between 0 and 15 ETH.	Must Have	Implemented (Functional)
FR-05	Sale Type Filter	The system shall allow filtering by sale mechanism: All, Fixed Price listings only, or Timed Auction listings only.	Should Have	Implemented (Functional)
FR-06	Verified Only Filter	The system shall provide a toggle to restrict displayed listings to verified creators only.	Should Have	Implemented (Functional)

FR-07	Catalog Sorting	The system shall allow sorting by Recently Added, Price: Low to High, Price: High to Low, and Most Popular (likes).	Must Have	Implemented (Functional)
FR-08	NFT Detail Page	The system shall provide a dedicated view showing high-resolution artwork, creator/owner cards, token ID, standard, description, traits, and history.	Must Have	Implemented (Functional)
FR-09	Favorite / Like	The system shall allow users to toggle favorites on any NFT, updating like count and persisting liked IDs in local storage.	Should Have	Implemented (Functional)

FR-10	Wallet Connection	The system shall provide an interactive modal simulating connection for MetaMask, WalletConnect, Coinbase, Phantom, and Rainbow.	Must Have	Implemented (Simulated)
FR-11	Network Switching	The system shall allow users to toggle active blockchain context between Ethereum, Polygon, Arbitrum, and Solana.	Should Have	Implemented (Simulated UI)
FR-12	Test Faucet	The system shall provide a 1-click test faucet in Navbar and modals that credits +5.0 simulated ETH to the user's test balance.	Must Have	Implemented (Functional)

FR-13	Artwork Upload	The system shall allow creators to upload images via file dialog or drag-and-drop, converting files to base64 Data URLs with live preview.	Must Have	Implemented (Functional)
FR-14	Minting Studio	The system shall validate title, description, price, category, blockchain, and royalty (0-20%) before minting new tokens.	Must Have	Implemented (Functional)
FR-15	Trait Builder	The system shall allow creators to dynamically add and remove custom trait_type and value metadata attribute pairs.	Should Have	Implemented (Functional)
FR-16	Instant Checkout	The system shall calculate total cost (price + gas), check balance sufficiency, deduct balance, reassign	Must Have	Implemented (Functional)

		owner, and log transaction.		
FR-17	Auction Bidding	The system shall validate bids > highest bid, verify balance, update highest bid, record bidder info, and log bid activity.	Should Have	Implemented (Functional)
FR-18	Make Offer	The system shall allow users to propose custom offer amounts and expiration durations (1, 3, 7 days) and log offer activities.	Should Have	Implemented (Functional)
FR-19	Profile Management	The system shall display collected, created, favorited NFTs, portfolio value, and allow editing profile name, bio, and avatar.	Should Have	Implemented (Functional)

FR-20	Data Reset	The system shall provide a 1-click control to clear localStorage and restore default marketplace demo datasets.	Could Have	Implemented (Functional)

12. User Stories

To ensure development remained user-centric, requirements were translated into standard Agile user stories mapped to target personas:

- **US-01:** As a prospective buyer, I want to browse digital artworks in an organized grid, so that I can discover new art and creators easily.
- **US-02:** As a collector with a budget, I want to filter listings by price range and sale format, so that I only see items I can afford to buy.
- **US-03:** As an art enthusiast, I want to search by artist name or keyword, so that I can quickly locate specific artworks I have seen elsewhere.
- **US-04:** As a cautious buyer, I want to inspect smart contract token specs and creator verification, so that I know the asset is authentic before buying.
- **US-05:** As a collector, I want to review historical sales and ownership transfers on an item, so that I can evaluate its market liquidity and provenance.
- **US-06:** As a Web3 newcomer, I want to connect my wallet with a simple visual selector, so that I do not have to struggle with complex technical configurations.
- **US-07:** As a student / tester, I want to claim free test ETH from a faucet, so that I can test buying and minting without risking real money.
- **US-08:** As an independent artist, I want to drag and drop my artwork into a minting tool, so that I can see an instant visual preview of my NFT.

US-09: As a creator, I want to define a creator royalty percentage up to 20%, so that I can receive automated compensation on secondary market resales.

US-10: As a creator, I want to add custom rarity traits (e.g., Background, Rarity Tier), so that my NFT adheres to standard metadata conventions.

US-11: As a buyer, I want to see an upfront line-item breakdown of item price and estimated gas before paying, so that I am never surprised by hidden fees.

US-12: As an auction participant, I want to place bids with 1-click increment buttons, so that I can compete in timed auctions quickly.

US-13: As a prospective buyer, I want to make an off-market offer with an expiration date, so that the seller can consider my proposal if the price is right.

US-14: As a collector, I want a personal profile showing my collected NFTs and total portfolio valuation, so that I can manage my digital asset inventory.

13. Acceptance Criteria

To establish unambiguous definitions of 'done' for engineering implementation, explicit acceptance criteria were authored for the primary user flows:

13.1 Marketplace Discovery & Catalog Filtering

- Given a user on the Explore page, when selecting a category pill, then only NFTs matching that category must be displayed.
- Given an active search query, when the user types in the search bar, then results must update instantly without requiring page refresh.
- Given the price slider, when adjusted to X ETH, then all items priced above X ETH must be filtered out.
- Given active filters resulting in 0 matches, then a friendly 'No NFTs Found' state with a 'Reset Filters' button must appear.

13.2 NFT Detail & Provenance

- Given an NFT detail page, the system must display high-res artwork, category tag, network badge, creator card, and current owner card.

- When the user clicks the 'Favorite' heart icon, the like count must increment/decrement immediately and persist across browser reloads.
- When clicking 'History' tab, the chronological log of Minted, Listed, Sold, and Bid transactions must be visible with addresses and dates.
- When clicking the contract address, the address must be copied to clipboard and trigger a confirmation toast alert.

13.3 Wallet Connection & Faucet

- When an unauthenticated user clicks 'Connect Wallet', the modal must present MetaMask, WalletConnect, Coinbase, Phantom, and Rainbow.
- When a wallet is selected, the modal must display a brief connection indicator and update the Navbar with truncated address and balance.
- When the user clicks '+5 ETH Faucet' in the Navbar or checkout modal, the balance must increment by exactly 5.0 ETH.

13.4 Creator Studio & Minting

- The minting form must require NFT Title, Description, Listing Price (> 0), and an uploaded artwork or chosen preset.
- If any mandatory field is missing, inline validation error messages in red must prevent form submission.
- The creator must be able to dynamically add trait rows and delete trait rows.
- Upon submission, the system must check gas sufficiency (~0.012 ETH), deduct gas, prepend the new NFT to the global catalog, and render a success screen.

13.5 Instant Checkout / Buy Now

- The checkout modal must display artwork preview, item price in ETH and USD, estimated gas (~0.0035 ETH), and creator royalty percentage.
- If the user's wallet balance is less than total cost, the 'Confirm & Pay' button must be disabled and display an 'Insufficient Funds' warning with a 1-click faucet button.
- Upon confirmation, the system must deduct total cost, reassign ownership to the active user, prepend a 'Sold' entry into transaction history, log user activity, and display a success toast.

13.6 Auction Bidding & Offers

- In the bid modal, the bid amount must default to at least current highest bid + 0.1 ETH.
- Any bid entered equal to or below the current highest bid must be rejected with an inline validation error.
- Submitting a bid must update highestBid on the NFT, record the bid in the bids tab, and log 'Bid Placed' in user activity.

14. Non-Functional Requirements

Non-functional requirements specify qualitative benchmarks regarding how the system should operate across usability, performance, reliability, and security:

14.1 Usability & Responsive UI

- **Cross-Device Responsiveness:** The application must render fluidly across mobile (<640px), tablet (640px–1024px), and desktop (>1024px) viewports using responsive grid layouts.
- **Micro-Interactions:** Interactive buttons, cards, and modal triggers must provide clear hover, active, focus, and disabled states.
- **Visual Contrast:** High contrast typography (white/gray on dark surface-950 background) ensuring readability for digital art inspection.

14.2 Performance & Latency

- **Client-Side Latency:** Catalog filtering, sorting, and keyword searching must execute within <100ms in-browser without UI stutter or unneeded network overhead.
- **Build Footprint:** The production singlefile bundle must compile in <3 seconds via Vite and load under 500KB gzipped.
- **Image Resilience:** Image cards must gracefully handle image load failures with automatic fallbacks to reliable placeholder assets.

14.3 Security & Trust Communication

- **Transaction Transparency:** All transactional flows must clearly communicate what permissions are being granted and what funds are being transferred.
- **Secure Link Protocols:** External links (e.g., block explorer links) must include rel='noreferrer' and target='_blank' to protect user privacy.
- **Zero Custody:** Prototype code must never request private keys or actual seed phrases under any circumstances.

14.4 Reliability & State Persistence

- **Local Storage Sync:** All user mutations (minted items, purchased items, updated profile data, favorites, bid history) must automatically sync to browser localStorage.
- **Error Resilience:** If corrupted data is detected in localStorage, the application must catch JSON parsing errors gracefully and fall back to clean default mock data.

15. UX / UI Requirements & Screen Architecture

PixelMint's user interface is built on a dark-mode Web3 design system utilizing Tailwind CSS, glassmorphism card containers, and neon purple/cyan accent gradients. Below is the structural architectural breakdown of each primary screen in the application:

15.1 Screen 1: Marketplace Landing Page (HomePage.tsx)

- Market Pulse Ticker: Top horizontal ribbon displaying live market indicators: ETH Gas (18 Gwei), 24h Volume (2,481 ETH), and Active Collectors.

- Hero Section: High-impact typography ('Discover & Own Extraordinary Digital Masterpieces'), primary CTA buttons ('Explore Marketplace' and 'Mint Artwork'), and live metric badges (12.5K+ Minted, 48K+ Collectors, 245K ETH Traded).
- Interactive Showcase Card: Right-hand column displaying featured drop card ('Abstract Genesis') with floating preview cards, live auction badges, and instant buy triggers.
- Curated Drops Section: 4-column grid showcasing featured artworks with quick buy overlays.
- Top Collections Leaderboard: Summary grid showing top 6 collections with floor prices, volume, and 24h gain/loss percentages.
- Educational 'How PixelMint Operates': 4-step card flow explaining Wallet Connection, On-Chain Minting, Instant Checkout, and Secondary Royalties.

15.2 Screen 2: Explore & Marketplace Catalog (ExplorePage.tsx)

- Header & Search Bar: Prominent search input with clear button and real-time query filtering.
- Quick Controls Bar: Sort dropdown (Recently Added, Price Low-High, Price High-Low, Popular), Advanced Filters toggle button with active indicator dot, and Desktop 3 vs 4 column grid layout switcher.
- Category Filter Pills: Horizontal scrolling pills for All, Art, Collectibles, Photography, Music, Gaming, Virtual Worlds.
- Advanced Parameters Drawer: Collapsible panel with price range slider (0–15 ETH), sale type toggle (Fixed vs Auction), verified creator checkbox, and 'Reset All' button.
- Dynamic Results Grid: Responsive card grid rendering NFTCard.tsx components with title, creator avatar, verified badge, price, like count, and format tags.

15.3 Screen 3: NFT Detail View (NFTDetailPage.tsx)

- Sticky Artwork Viewport: Left column featuring large framed artwork container, live auction countdown badge, and high-fidelity rendering.

- Asset Header: Category pill, network indicator, favorite heart button, link copy share button, and Etherscan block explorer link.
- Creator & Owner Cards: Dual cards showing avatar, name, verification status, and public wallet address.
- Pricing & Action Box: Prominent price display in ETH and estimated USD, paired with contextual primary action ('Buy Now' vs 'Place Bid') and secondary action ('Make Offer'). If the active user owns the artwork, 'Buy Now' is disabled with the label 'You Own This Artwork'.
- Segmented Tabs Panel: Three view tabs: 'Details' (description, on-chain trait attribute cards, token ID, ERC-721 standard, royalty split, contract copy button), 'History' (chronological event stream), and 'Bids' (bidder activity log).

15.4 Screen 4: Creator Minting Studio (CreatePage.tsx)

- Left Upload Column: Drag-and-drop file upload zone supporting PNG, JPG, WebP with live preview and change overlay. Below it, a 3x2 grid of curated generative artwork sample presets allows 1-click visual selection for quick testing.
- Creator Identity Card: Visual card confirming the user's active wallet address, avatar, and minting role.
- Right Configuration Column: Form inputs for NFT Title, Description textarea, Category dropdown, Target Blockchain selector (Ethereum, Polygon, Arbitrum, Solana), Listing Format toggle (Fixed Price vs Timed Auction), Price input, and Creator Royalty slider (0–20%).
- Dynamic Trait Builder: Repeatable trait inputs allowing creators to specify custom trait_type and value pairs with delete controls.
- Gas Fee & Security Summary: Estimated network gas card (~0.012 ETH), smart contract reference (PixelMintERC721.sol), current wallet balance, and contextual +5 ETH faucet link if balance is low.
- Mint Confirmation View: Post-minting modal celebrating deployment with artwork thumbnail, category badge, and buttons to view NFT details or mint another asset.

15.5 Screen 5: Interactive Modals

- BuyModal.tsx: Instant checkout modal with artwork thumbnail, cost breakdown (Price + Gas + Royalty = Total), balance verification, 1-click test faucet, and 3-step progress states (Checkout -> Processing -> Confirmed).
- BidModal.tsx: Timed auction bidding modal with current bid display, auction end time, dynamic increment buttons (+0.1, +0.5, +1.0 ETH), and balance validation.
- MakeOfferModal.tsx: Direct negotiation modal with offer amount input in ETH and expiration dropdown (1 day, 3 days, 7 days).
- WalletModal.tsx: Multi-wallet provider modal listing MetaMask, WalletConnect, Coinbase, Phantom, and Rainbow with connection spinners and encrypted protocol badge.
- EditProfileModal.tsx: Profile customization modal allowing editing display name, handle, bio, email, avatar URL, and banner URL.

16. Edge Cases and Error States

A key indicator of thorough product management is anticipating what happens when users deviate from the happy path. The following table documents 10 realistic edge cases, how the system is specified to behave, and its current prototype handling:

Edge Case Scenario	Trigger Condition	Expected System Behavior	Prototype Implementation Handling

Disconnected Wallet on Action	User clicks Buy, Bid, or Offer without connecting a wallet first.	System intercepts action, halts transaction, and automatically launches WalletModal.	Fully implemented. Automatically opens WalletModal.tsx and halts checkout.
Insufficient ETH for Purchase	User wallet balance is lower than (NFT Price + 0.0035 ETH Gas).	System disables confirm button, displays red warning badge, and offers 1-click +5 ETH Faucet.	Fully implemented. Shows exact balance deficit and displays '+5 ETH Faucet' button.
Insufficient ETH for Minting Gas	Creator has less than ~0.012 ETH required for simulated contract deployment.	System displays gas warning and blocks submission until balance is replenished via faucet.	Fully implemented in CreatePage.tsx with inline faucet trigger.
Self-Purchase Attempt	User navigates to an NFT that is currently owned by	Primary buy button is disabled with label 'You Own This Artwork'	Fully implemented in NFTDetailPage.tsx (checks

	their own wallet address.	to prevent self-trading.	nft.owner.address === user.address).
Bid Lower Than Current Highest	Bidder submits a bid equal to or less than current highest bid.	System rejects bid, renders red inline error message, and enforces minimum +0.1 ETH increment.	Fully implemented in BidModal.tsx with real-time validation.
Missing Required Form Fields	Creator submits minting form without title, description, price, or image.	System halts submission, highlights missing fields in red, and displays specific field errors.	Fully implemented with comprehensive validate() form checker in CreatePage.tsx.
Zero Matching Search Results	User enters a query or price range that matches zero catalog items.	System displays centered 'No NFTs Found' empty state with 'Reset All Filters' CTA button.	Fully implemented in ExplorePage.tsx.

Broken Image Source	Remote Unsplash or CDN image URL fails to load due to network error.	Image tag onError handler catches failure and replaces src with fallback artwork.	Fully implemented on image components across detail and catalog cards.
User Cancels Mid-Transaction	User closes modal while reviewing checkout or bidding details.	System aborts flow cleanly, resets modal step, and leaves user balance and state unchanged.	Fully implemented across all modals.
Wallet Disconnection During Session	User disconnects wallet from Navbar dropdown while on an authenticated page.	System updates global state, displays notification toast, and renders clean connect-wallet empty state.	Fully implemented on CreatePage and ProfilePage.

17. Proposed Success Metrics - Not Currently Measured

In accordance with product management rigor, this section defines a proposed telemetry and success metric framework for PixelMint. Because PixelMint is currently a student portfolio prototype, NO live production analytics or user

conversion tracking have been executed. The metrics below represent what should be instrumented prior to a live testnet or mainnet launch.

Telemetry Disclaimer: STATUS: PROPOSED METRIC FRAMEWORK - NOT CURRENTLY MEASURED. No live production user tracking, telemetry, or analytics platforms (e.g., Mixpanel, Google Analytics, PostHog) are active in the current prototype. All values below represent target KPIs and hypotheses for future testing.

17.1 Proposed Acquisition & Discovery Metrics

- **Catalog Exploration Rate (Target: >65%):** Percentage of unique landing page visitors who navigate to the Explore catalog or search bar within their first session.
- **Search & Filter Engagement:** Frequency of search bar queries, category pill clicks, and filter drawer adjustments per active session.
- **Detail View Throughput:** Number of unique NFT detail view sessions per visitor, identifying high-demand artworks and categories.

17.2 Proposed Funnel & Conversion Metrics

- **Wallet Connection Completion Rate:** Percentage of users who initiate wallet connection in WalletModal.tsx and successfully complete authentication.
- **Checkout Conversion Funnel:** Percentage of users who click 'Buy Now' and successfully reach the 'Purchase Confirmed' modal state.
- **Checkout Abandonment Rate:** Percentage of users who open BuyModal.tsx but exit before clicking confirm, specifically tracking exits caused by insufficient balance.
- **Auction Bid Conversion Rate:** Percentage of users who open BidModal.tsx and submit a qualifying competitive bid.

17.3 Proposed Creator Studio Metrics

- **Minting Studio Funnel Completion:** Percentage of users who click 'Create' and successfully complete all steps to deploy a minted NFT.

- **Time-to-Mint Efficiency:** Average elapsed time from entering CreatePage.tsx to completing metadata entry and minting submission (Target: <3 minutes).
- **Royalty Preference Distribution:** Distribution of creator royalty percentages selected on the slider (evaluating whether creators prefer 5%, 10%, or 20%).

18. Risks and Mitigations

Building decentralized applications involves unique technical, economic, and behavioral risks. The matrix below outlines the primary risks identified for PixelMint, their potential severity, proposed mitigations, and their status in the prototype:

Risk Identified	Risk Category	Severity	Proposed Mitigation Strategy	Current Prototype Status
Smart Contract Exploits	Security / Technical	Critical	Use battle-tested OpenZeppelin ERC-721 contracts, formal code audits, and bug bounty programs prior to mainnet.	Mitigation Not Implemented. Prototype uses client-side state simulation.

Network Gas Fee Spikes	Economic / UX	High	Deploy marketplace contracts to Layer-2 rollups (Arbitrum, Optimism, Polygon) and implement EIP-2771 gasless relayers.	Open Consideration. Prototype displays simulated static gas (~0.0035 ETH).
Wallet UX Confusion	Usability / Adoption	High	Provide plain-language transaction explanations, safe test faucet, and friendly error guidance.	Partially Mitigated via test faucet, multi-wallet selector, and clear modals.
Copyright / IP Infringement	Legal / Content	High	Implement community reporting, creator identity verification, and automated image hash matching to	Mitigation Not Implemented. Prototype includes visual 'Verified' badges only.

			detect duplicates.	
Client Storage Data Loss	Technical / Reliability	Medium	Migrate from browser localStorage to persistent cloud databases (PostgreSQL) and decentralized IPFS pinning.	Prototype uses localStorage with clean 'Reset Demo Data' fallback.
Regulatory & Compliance Uncertainty	Legal / Regulatory	Medium	Ensure tokens represent utility digital art rather than financial securities; maintain terms of service and disclaimers.	Documented in PRD assumptions and portfolio disclaimers

19. Product & Technical Assumptions

Every product requirements document rests on foundational assumptions. Documenting these assumptions ensures stakeholders understand the operating premises under which PixelMint was designed:

1. **User Domain Familiarity:** We assume target users have access to a modern desktop or mobile web browser and understand the basic concept that

digital assets can be uniquely owned and traded on a blockchain ledger.

2. **Web3 Wallet Accessibility:** We assume users either already own a non-custodial crypto wallet (MetaMask, Phantom, etc.) or are willing to connect one using standard Web3 provider protocols.
3. **Transaction Cost Acceptance:** We assume users understand that blockchain transactions require network fees ('gas') paid in cryptocurrency, and that fees vary based on network congestion.
4. **Creator Royalty Motivation:** We assume digital creators value secondary resale royalties and are motivated to mint when provided with a clean, code-free interface.
5. **Technical Infrastructure Standard:** We assume Ethereum and compatible EVM chains remain the dominant standard for digital art provenance, while recognizing that Layer-2 rollups are essential for affordable consumer scaling.

20. Open Product Questions

A mature Product Manager does not pretend every strategic decision has already been finalized. Identifying unresolved questions demonstrates critical product thinking and establishes an agenda for subsequent research:

1. **Primary Blockchain Deployment Architecture:** Should PixelMint deploy primarily to Ethereum Mainnet for prestige and high-value 1-of-1 art drops, or should it launch natively on an EVM Layer-2 (such as Arbitrum One or Base) to reduce transaction fees for consumer creators? What is the user trade-off between Layer-1 collector prestige and Layer-2 fee affordability?
2. **Gasless Transaction Feasibility:** What specific technical architecture should be adopted to eliminate or subsidize gas fees for new users? Should PixelMint implement an account abstraction model (ERC-4337) with paymasters, or a Biconomy gasless meta-transaction relayer?
3. **Offer & Bid Escrow Mechanics:** Should off-market offers and auction bids be stored off-chain using cryptographic signatures (similar to OpenSea Seaport protocol) to save users gas fees until an offer is accepted, or should they be

locked on-chain in an escrow contract?

4. **Artist Verification & Content Governance:** What level of creator verification is appropriate? Does an open, permissionless minting model invite rampant spam and stolen art, or does a curated approval process create an elitist barrier that contradicts Web3 principles?
5. **Royalty Enforcement in Competitive Markets:** Should secondary creator royalties be strictly enforced via custom contract registries, or should PixelMint support optional collector tipping in light of broader marketplace fee wars?

21. Future Opportunities & AI-Assisted Creator Tools

As PixelMint evolves beyond its initial prototype, several high-impact product opportunities have been identified to expand creator capabilities and market differentiation.

21.1 AI-Assisted NFT Description Generation

One of the most promising future enhancements explored during product planning is an AI-assisted metadata generation workflow. Many visual artists excel at digital illustration but struggle to write compelling conceptual narratives, lore, and algorithmic process descriptions for their marketplace listings.

Intended Product Workflow

- **Step 1 (Upload):** Creator drags and drops custom digital artwork into the Creator Studio upload viewport.
- **Step 2 (Vision Analysis):** The system passes the artwork image to a multimodal vision model (e.g., Gemini Vision API) alongside creator-selected style tags (e.g., Cyberpunk, Surrealism, Generative).

- **Step 3 (Draft Generation):** The AI engine generates a draft conceptual description detailing visual themes, color palette symbolism, and potential artistic lore.
- **Step 4 (Review & Refinement):** The creator reviews the draft in an editable textarea, modifies tone, adds personal narrative context, and finalizes the text.
- **Step 5 (Minting):** The creator confirms the listing, and the edited description is permanently inscribed into the token's ERC-721 metadata.

Implementation Status of AI Description Tool: STATUS: FUTURE ROADMAP / NOT IMPLEMENTED IN CURRENT CODEBASE. Inspection of the project codebase confirms that no AI APIs, Python backend models, or text generation hooks are active in the current prototype. This feature is documented here purely as a strategic product opportunity for Phase 3 development.

21.2 Additional Strategic Opportunities

- **Batch Minting & Generative Drop Engine:** Allow creators to upload multi-asset collections (e.g., 50–100 generative variations) with automated ERC-1155 or ERC-721A batch minting, drastically reducing cumulative deployment costs.
- **Fiat Credit Card Checkout:** Partner with Web3 payment processors (Stripe Crypto, MoonPay) to enable collectors to purchase digital assets directly using credit cards, with the backend automatically provisioning non-custodial embedded wallets (e.g., Privy or Dynamic).
- **Physical-Digital ('Phygital') Redeemables:** Integrate physical redemption mechanisms where token holders can claim physical museum-grade acrylic prints of their digital art via cryptographic signature verification.

22. Implementation Status & Feature Audit

This section represents a comprehensive, transparent audit of PixelMint's actual

codebase. It compares product design intentions against the reality of the software currently running in the workspace.

Feature Area	Expected Product Behavior	Current Code Implementation	Verified Status	Technical Assessment & Notes
Catalog Browsing	Browse assets with dynamic card rendering and paginatio n.	ExplorePage.tsx renders items from MarketplaceCon text state.	Implemen ted	Fully functional. Responsive layout with 3 vs 4 column toggle.
Keyword Search	Search titles, artists, and collection s with instant matching.	Instant filter across nfts array with suggestions dropdown in Navbar.	Implemen ted	Fully functional client-side search. Instant UI response.
Parametric Filters	Filter by category, price slider, sale format,	useMemo filter hook filtering context nfts array.	Implemen ted	Fully functional. Includes 'Reset All' button and drawer.

	and verification.			
Item Detail View	Inspect artwork, provenance, traits, description, and history.	NFTDetailPage.tsx dynamically pulls by ID with 3 view tabs.	Implemented	Fully functional. Includes clipboard copy and fallback image.
Simulated Wallet	Connect Web3 wallet provider and display truncated address.	WalletModal.tsx simulates connection with 700ms timeout.	Partially Implemented (UI Sim)	Does not invoke window.ethereum or real browser extensions.
Test Faucet	Add test funds to wallet for checkout testing.	addFunds(5.0) increments user.balance in context & localStorage.	Implemented	Fully functional test utility. Accessible via Navbar & modals.
Creator Studio	Upload image, define metadata, traits, royalties,	CreatePage.tsx handles file reader, validation, and mintNFT().	Implemented	Fully functional. Stores base64 / presets in state & localStorage.

	and mint.			
Instant Buy Checkout	Check balance, estimate gas, reassign owner, log transaction.	BuyModal.tsx executes buyNFT() with balance checks and gas fee.	Implemented	Fully functional simulation. Correctly updates ownership.
Auction Bidding	Validate bid > current, update highestBid, log activity.	BidModal.tsx validates bid and updates NFT bids array.	Implemented	Fully functional. Validates balance and minimum increment.
Make Offer	Submit off-market proposal with expiration duration.	MakeOfferModal.tsx validates amount and logs user activity.	Partially Implemented	Logs offer activity and toast; does not notify seller or hold escrow.
Profile Dashboard	Inspect owned, created, favorited NFTs, valuation,	ProfilePage.tsx filters nfts by user address and liked IDs.	Implemented	Fully functional. Includes EditProfileModal.tsx.

	and edits.			
Collections Board	Rank collections by floor, volume, 24h change, and timeframe.	CollectionsPage.tsx displays ranking table with time filter.	Implemented	Fully functional. Clicking navigates to filtered Explore.
Smart Contracts	Deploy and execute ERC-721 contracts on EVM testnet.	No Solidity files (.sol) or hardhat/foundry configs exist.	Not Implemented (Simulated)	Client prototype simulates contracts using pseudo hashes.
Security Auditing	Third-party formal verification of contract security.	Landing page badge states '100% Audited' as UI placeholder.	Not Implemented (UI Only)	No formal audit was performed. Must be clearly acknowledged.
Decentralized IPFS	Pin media and metadata JSON to IPFS or Arweave	Files stored in memory and localStorage as base64 Data URLs.	Not Implemented / Future	Decentralized storage pipeline reserved for Phase 2.

	nodes.			
AI Description Tool	Generate draft artwork descriptions using vision AI.	No AI dependencies, endpoints, or scripts exist in repo.	Not Implemented (Future)	Documented as Phase 3 feature opportunity.

23. Product Roadmap

To illustrate structured product planning, a phased roadmap is outlined below. It transitions PixelMint from its current interactive prototype into a fully decentralized, production-grade application.

Phase 1: Interactive Client Prototype (Current MVP — Completed)

- Design and build core consumer user flows: Catalog Explore, Real-time Search, Dynamic Filters, and Item Details.
- Implement client-side Creator Studio with drag-and-drop file upload, royalty slider (0-20%), and custom trait builders.
- Develop simulated transactional engine: multi-wallet connection modal, test faucet (+5 ETH), instant buy checkout, and auction bidding.
- Establish local state persistence via browser localStorage to ensure continuity during testing and portfolio demonstration.
- Author complete Product Requirements Document (PRD) documenting user

problems, architecture, and feature audit.

Phase 2: Live Testnet Integration & Web3 Infrastructure (Next Iteration)

- Author and test secure OpenZeppelin ERC-721 smart contracts on Ethereum Sepolia testnet with ERC-2981 royalty standards.
- Integrate injected Web3 provider libraries (wagmi / viem / RainbowKit) for live MetaMask and WalletConnect execution.
- Implement decentralized storage pipeline using Pinata / Infura IPFS APIs for permanent media and JSON metadata hosting.
- Deploy automated orderbook contract on testnet supporting true escrow-backed offers and reserve-price timed auctions.
- Deploy to Layer-2 testnet (Arbitrum Sepolia or Base Sepolia) to evaluate real-world gas fee reductions.

Phase 3: Production Hardening, Creator Tools & AI Integration (Future)

- Conduct professional smart contract security audit with independent Web3 security firm prior to mainnet deployment.
- Build and deploy AI-assisted metadata generator using Gemini Vision API to help creators draft conceptual artwork descriptions.
- Integrate fiat on-ramp gateway (Stripe Crypto / MoonPay) allowing non-crypto users to purchase NFTs using standard debit/credit cards.
- Implement creator collection launchpad supporting ERC-721A batch generative drops and custom minting allowlists.

24. Conclusion & PM Retrospective

The process of creating and keeping records of PixelMint has been a highly beneficial learning exercise in the field of product management. Initially, when I started the project as my university capstone, my main concern was the novel aspect of owning digital art and the mechanics of a decentralized marketplace. But after I had worked on designing user journeys and examined the points at which ordinary users encounter difficulties, I came to the conclusion that the main obstacle to Web3 adoption is not blockchain throughput - it is rather user

experience empathy.
During the course of this project I gained the ability to make purposeful decisions regarding product trade-offs, choosing to focus on upfront cost transparency and safe sandbox simulation rather than on complicated and poorly developed smart contract integrations. The act of documenting this PRD helped to clarify the important difference between product requirements and technical implementation, and it showed me that it is much more valuable to carry out an honest and self-aware evaluation of what is actually working and what is merely being simulated than it is to create a false impression of having completed features.
PixelMint shows off my capability in identifying user pain points, in defining the target personas, in mapping out thorough user journeys, in setting clear boundaries for the MVP, in writing strict functional requirements, in anticipating realistic edge cases, and in planning phased evolutionary roadmaps. These fundamental Product Management principles will serve as the basis for my work when I am an Associate Product Manager or Junior PM in the development of well-thought-out, consumer-focused digital products.

25. Appendix

Appendix A: Technical Architecture & Repository Overview

PixelMint is built as a single-page application (SPA) using modern front-end technologies. The table below summarizes the core technical components verified during codebase inspection:

Component	Technology / Package	Version	Role in Application
Core Framework	React + TypeScript	19.2.3 / 5.9.3	Component hierarchy, type safety, state management

Build Tooling	Vite	7.2.4	Rapid development bundling, singlefile production compiling
Styling Engine	Tailwind CSS v4	4.1.17	Custom glassmorphism design system, dark mode surface palette
Iconography	Lucide React	0.577.0	Clean iconography across navigation, actions, and status badges
State Architecture	React Context API	Native React 19	Global marketplace state, wallet state, toasts, modals
Data Persistence	Browser localStorage	HTML5 Standard	Persists NFTs, collections, user profile, likes, activities

Appendix B: Web3 & NFT Terminology Glossary

- **NFT (Non-Fungible Token):** Non-Fungible Token. A unique digital identifier recorded on a blockchain that certifies ownership and authenticity of a specific asset.
- **ERC-721 Standard:** The smart contract technical standard on the Ethereum blockchain for non-fungible digital assets, ensuring interoperability.
- **Minting:** The process of validating and inscribing a digital asset onto a blockchain ledger, establishing its initial timestamped provenance.
- **Gas Fee:** Computational execution fees paid to blockchain network validators/minters to process and record transactions.
- **Creator Royalty:** An automated percentage of secondary market resales paid directly to the original creator on secondary transfers (standardized by ERC-2981).

- **Floor Price:** The lowest currently listed fixed price for an NFT within a specific collection, serving as an entry valuation benchmark.
- **Web3 Wallet:** A digital tool (hardware or software) that manages private keys and allows users to sign blockchain transactions and interact with decentralized apps.
- **Layer-2 Rollup:** Secondary blockchain networks built on top of Layer-1 (like Ethereum) that batch transactions to achieve higher throughput and lower fees.
- **IPFS:** InterPlanetary File System. A peer-to-peer decentralized storage protocol used to store NFT media files and JSON metadata securely off-chain.